

Python 编程与人工智能

机器学习

Chengwei Qin@人工智能学域

Scikit-Learn 机器学习基础

- 加载数据集、划分数据、预处理数据

Scikit-Learn 中的模型

- 监督学习模型、无监督学习模型

模型评估与选择

- 泛化：偏差-方差权衡
- 数据划分协议、评估指标、交叉验证

超参数优化

- 搜索算法、搜索空间



Scikit-Learn 机器学习基础

scikit-learn 是一个用于机器学习的开源 Python 库。
它已成为 Python 通用机器学习的“事实标准”。

核心特性

- 坚实基础：依托 NumPy、Pandas、SciPy 和 matplotlib 保证性能与可视化。
- 覆盖全面：包含分类、回归、聚类、降维等多种算法。
- 文档完善：在线文档出色，示例丰富。
- 可商用：以宽松的 BSD 许可证开源。

Scikit-learn 中的大多数机器学习项目遵循以下一般步骤。

- **加载数据：**加载数据集，将特征 ('X') 与标签 ('y') 分开。
- **划分数据：**将数据划分为训练集和测试集。
- **预处理（可选）：**构建预处理流水线（例如缩放特征）。
- **训练模型：**创建模型实例，并用 '.fit()' 方法在训练数据上训练。
- **评估模型：**用训练好的模型通过 '.predict()' 在测试集上做预测，并评估其性能。

加载数据集

scikit-learn 自带若干玩具数据集，并提供获取更大规模真实数据集的辅助工具。

```
from sklearn.datasets import load_iris

iris = load_iris()
X = iris.data # Features
y = iris.target # Labels

print(X.shape, y.shape) # (150, 4) (150,)
```

加载玩具数据集（例如 Iris）

```
from sklearn.datasets import fetch_openml

mnist = fetch_openml('mnist_784', version=1)
X, y = mnist["data"], mnist["target"]

print(X.shape, y.shape) # (70000, 784) (70000,)
```

获取真实数据集（例如 MNIST）

加载数据后，下一步是将其划分为训练集和测试集。这样便可在一部分数据上训练模型，并在另一部分未见过的数据上评估其泛化性能。

'train_test_split' 函数

```
from sklearn.model_selection import train_test_split

# Assume X and y are already loaded

# test_size=0.2 means 20% for testing, 80% for training
# random_state ensures the split is reproducible
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42
)

print("Training data shape:", X_train.shape)
print("Testing data shape:", X_test.shape)
```

为什么要预处理数据？

例如，真实数据集中的特征往往在尺度、范围和类型上差异很大

未缩放数据的问题：

- 数值范围较大的特征可能主导模型。
- 基于梯度的算法（如神经网络）可能收敛很慢或陷入停滞。
- 基于距离的算法（如 KNN 或 K-Means）会产生有偏结果。

Example: House Price Prediction Dataset

- Age of the house: [0, 100] years
- Size of the house: [500, 5000] sqft
- Price of the house: [\$50,000, \$1,000,000]

'sklearn.preprocessing' 模块为常见数据变换任务提供丰富工具。

- 缩放
- 归一化
- 类别特征编码
- 离散化 / 二值化
- 缺失值填补
- 生成多项式特征
- 降维（位于 'sklearn.decomposition'）

特征缩放: StandardScaler

StandardScaler 是最常用的缩放器之一。

它通过去除均值并将特征缩放到单位方差来实现标准化。所得分布的均值为 0、标准差为 1

$$x' = \frac{x - \mu}{\sigma}$$

where μ is the mean and σ is the standard deviation of the feature column.

Important API Methods

- 'fit(X)': Computes the mean (μ) and standard deviation (σ) from the training data 'X'.
- 'transform(X)': Applies the learned scaling transformation to the data 'X'.
- 'fit_transform(X)': A convenient shortcut that performs both fitting and transforming in one step.

StandardScaler: 代码示例

```
from sklearn import preprocessing
import numpy as np

X_train = np.array([[ 1., -1.,  2.],
                    [ 2.,  0.,  0.],
                    [ 0.,  1., -1.]])

# 1. Initialize the scaler
scaler = preprocessing.StandardScaler()

# 2. Learn the scaling parameters from the training data
scaler.fit(X_train)

# 3. Apply the transformation to the data
X_scaled = scaler.transform(X_train)

print("Scaled Data:\n", X_scaled)
# [[ 0.          -1.22474487  1.33630621]
#  [ 1.22474487  0.          -0.26726124]
#  [-1.22474487  1.22474487 -1.06904497]]

# You can also do it in one step
X_scaled_again = preprocessing.StandardScaler().fit_transform(X_train)
```

其他常用缩放器

scikit-learn 还提供若干适用于不同场景的缩放器。

- 'MinMaxScaler': 将每个特征缩放到给定范围，通常为 [0, 1]
- 'RobustScaler': 使用对离群值稳健的统计量（中位数与四分位距）来缩放特征。若数据中有大量可能扭曲均值和方差的离群值，请使用它。

```
min_max_scaler = preprocessing.MinMaxScaler()
X_train_minmax = min_max_scaler.fit_transform(X_train)

# All values in X_train_minmax will be between 0 and 1
print(X_train_minmax)
```

归一化

归一化将各个样本（行）缩放到单位范数（长度为 1）。当特征向量的方向比其幅度更重要时，这一做法很有用。

```
X = [[ 1., -1.,  2.],
      [ 2.,  0.,  0.],
      [ 0.,  1., -1.]]
# fit() does nothing for Normalizer, but is needed for pipeline
# compatibility
normalizer = preprocessing.Normalizer().fit(X)
normalized_X = normalizer.transform(X)
print(normalized_X)
# [[ 0.40824829 -0.40824829  0.81649658]
#   [ 1.          0.          0.          ]
#   [ 0.          0.70710678 -0.70710678]]
```

Scaler vs. Normalizer

- **Scalers** operate on columns (features).
- **Normalizers** operate on rows (samples).

大多数机器学习模型只接受数值数据。
类别特征（如 'male'/'female' 或国家名称）必须转换为数字。

OrdinalEncoder

- 将类别特征编码为整数数组。
- 最适合顺序有意义的有序数据（例如低、中、高）。

```
from sklearn.preprocessing import OrdinalEncoder

# Data has a clear order: low < medium < high
X_ordinal = [['low'], ['medium'], ['high'], ['low']]
encoder = OrdinalEncoder(categories=[['low', 'medium', 'high']])
print(encoder.fit_transform(X_ordinal))
# [[0.], [1.], [2.], [0.]]
```

对于没有顺序的名义特征（如国家名称），通常优先使用 OneHotEncoder，以免模型假定虚假的顺序。

离散化（KBinsDiscretizer）

离散化（或分箱）将连续特征变换为离散区间（箱）。这有时有助于难以处理连续数据的模型。

```
from sklearn.preprocessing import KBinsDiscretizer

X = np.array([[ -3.,  5., 15],
               [  0.,  6., 14],
               [  6.,  3., 11]])

# n_bins: number of bins for each feature
# encode: 'ordinal' returns an integer for each bin
est = KBinsDiscretizer(n_bins=[3, 2, 2], encode='ordinal')
X_binned = est.fit_transform(X)

print(X_binned)
# [[0.  1.  1.]
#   [1.  1.  1.]
#   [2.  0.  0.]
```

■ 二值化 (Binarizer)

二值化是离散化的特例，根据阈值将特征转换为 0 或 1。

```
X = [[ 1., -1.,  2.],
      [ 2.,  0.,  0.],
      [ 0.,  1., -1.]]

# Values > 0 become 1, others become 0
binarizer = preprocessing.Binarizer(threshold=0.0)
X_binarized = binarizer.fit_transform(X)

print(X_binarized)
# [[1., 0., 1.],
#   [1., 0., 0.],
#   [0., 1., 0.]]
```

■ 缺失值填补：处理缺失值

数据集经常包含缺失值（表示为 'NaN'）。在把数据送入模型之前必须先填补。这一过程称为缺失值填补。

SimpleImputer

用简单策略替换缺失值，例如相应列的均值、中位数或最频繁值。

```
from sklearn.impute import SimpleImputer
import numpy as np

# Use 'mean' strategy to fill NaNs
imputer = SimpleImputer(missing_values=np.nan, strategy='mean')

X = [[np.nan, 2, 3], [4, np.nan, 6], [10, np.nan, 9]]
imputer.fit([[7, 2, 3], [4, np.nan, 6], [10, 5, 9]]) # Fit on data
                                                    without NaNs in a column

# Transform the data with missing values
print(imputer.transform(X))
# [[ 7.    2.    3. ]
#  [ 4.    3.5   6. ]
#  [10.    3.5   9. ]]
```

简单策略速度很快，但更复杂的方法常能借助特征间的关系得到更好的结果。

KNNImputer

- 用训练集中 k 近邻的均值填补缺失值。
- 计算开销可能较大，但效果很好。

```
1 from sklearn.impute import KNNImputer
2 nan = np.nan
3 X = [[1, 2, nan], [3, 4, 3], [nan, 6, 5], [8, 8, 7]]
4 imputer = KNNImputer(n_neighbors=2)
5 print(imputer.fit_transform(X))
6 # [[1, 2, 4], [3, 4, 3], [5.5, 6, 5], [8, 8, 7]]
```

生成多项式特征

- `PolynomialFeatures` 可以生成一个新特征矩阵，其中包含次数小于或等于指定次数的全部特征多项式组合。
- 这是为 线性回归 等模型引入非线性的有力方法。

```
from sklearn.preprocessing import PolynomialFeatures

X = np.arange(6).reshape(3, 2)
# X is [[0, 1], [2, 3], [4, 5]]

poly = PolynomialFeatures(degree=2)
X_poly = poly.fit_transform(X)

# For features (X1, X2), this generates (1, X1, X2, X1^2, X1*X2, X2^2)
print(X_poly)

# [[ 1.  0.  1.  0.  0.  1.]
#  [ 1.  2.  3.  4.  6.  9.]
#  [ 1.  4.  5. 16. 20. 25.]
```

用 PCA 降维

- 主成分分析（PCA）是广泛使用的降维技术。
- 它找出能够解释数据中最大方差的轴（主成分）。

```
from sklearn.decomposition import PCA
import numpy as np
X = np.array([[ -1, -1], [-2, -1], [-3, -2], [1, 1], [2, 1], [3, 2]])
pca = PCA(n_components=2)
pca.fit(X)

print(pca.explained_variance_ratio_)
# [0.99244289 0.00755711]
# The first component explains 99.2% of the variance!

print(pca.singular_values_)
# [6.30061232 0.54980396]
```

应用场景：

减少特征数量以加快训练。

将高维数据投影到 2 维或 3 维以便可视化。

缓解“维度灾难”



Scikit-Learn 中的模型

监督学习模型

- 线性回归
- 逻辑回归
- 支持向量机 (SVM)
- 决策树
- 多层感知机 (MLP)

无监督学习模型 (聚类)

- K-Means
- 凝聚聚类
- DBSCAN
- 谱聚类
- 高斯混合

线性回归是基础的回归算法，也是监督学习的良好起点，用于预测连续值。

- 它是“白盒”模型，相对容易解释。
- 它假设特征与目标变量之间存在线性关系。
- 模型通过寻找穿过数据的最佳拟合直线（或超平面）进行学习。

The Equation: The model predicts an output $\hat{y}$ based on a linear combination of input features $x_1, \dots, x_n$ with learned weights (coefficients) $\theta_1, \dots, \theta_n$ and an intercept (bias) θ_0 .

$$\hat{y} = \theta_0 + \theta_1 x_1 + \theta_2 x_2 + \dots + \theta_n x_n$$

```
from sklearn.linear_model import LinearRegression

# Assume X_train, y_train are loaded
lin_reg = LinearRegression()
lin_reg.fit(X_train, y_train)

# View the learned parameters
print("Intercept (theta_0):", lin_reg.intercept_)
print("Coefficients (theta_1...):", lin_reg.coef_)

# Make a prediction
some_data = X_test[0].reshape(1, -1)
prediction = lin_reg.predict(some_data)
print("Prediction:", prediction)
```

为什么不用 线性回归 做分类？

若试图用一条直线分隔类别（例如将“良性”记为 0、“恶性”记为 1）：

- 模型可能预测出 0-1 范围之外的值（例如 -0.5 或 1.5），这些值作为概率没有意义。
- 决策边界（例如取在 0.5）对离群值可能非常敏感。

结论： 我们需要专为分类设计、并能输出概率的模型，例如 逻辑回归。

逻辑回归是二分类与多分类问题的基线模型。

它通过在输出上加入“压缩”函数，对线性回归加以改造。

- 1 Calculate a score using a linear equation: $z = \theta^T x = \theta_0 + \theta_1 x_1 + \dots$
- 2 Pass this score through a **Sigmoid function** to transform it into a probability.

逻辑回归中的 Sigmoid 函数

sigmoid 函数 $\sigma(z)$ 将任意实数映射到区间 $(0, 1)$ ，因而适合表示概率。

Function and Properties

$$\sigma(z) = \frac{1}{1 + e^{-z}}$$

- As $z \rightarrow \infty$, $\sigma(z) \rightarrow 1$.
- As $z \rightarrow -\infty$, $\sigma(z) \rightarrow 0$.
- If $z = 0$, $\sigma(z) = 0.5$. This is the decision boundary.

以二分类为例

- 完整模型计算样本属于正类（class 1）的概率
- 决策规则：若 $p \geq 0.5$ ，预测 class 1；若 $p < 0.5$ ，预测 class 0

$$\hat{p} = \sigma(\theta^T x)$$

```
from sklearn.datasets import load_iris
from sklearn.linear_model import LogisticRegression

# Load data and use two features for visualization
iris = load_iris()
X = iris.data[:, :2]
y = iris.target

# Logistic Regression handles multiclass classification
# 'C' is the inverse of regularization strength
softmax_reg = LogisticRegression(solver="lbfgs", C=10)
softmax_reg.fit(X, y)

# Predict the class for a new flower
new_flower = [[5, 2]]
print("Predicted class:", softmax_reg.predict(new_flower))

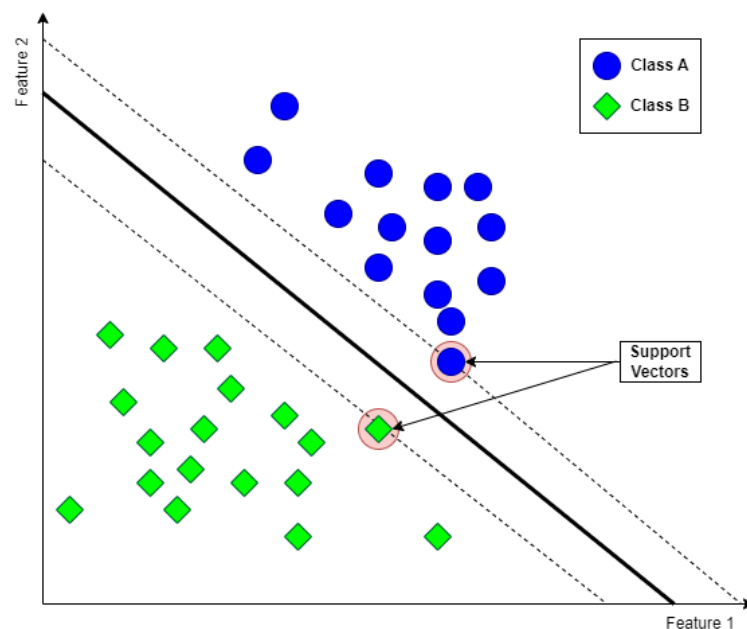
# Predict the probabilities for each class
print("Probabilities:", softmax_reg.predict_proba(new_flower))
```

支持向量机 (SVM)

支持向量机 (SVM) 是一种强大且通用的模型，能够进行线性或非线性分类、回归，甚至异常检测，尤其适合对复杂的中小规模数据集进行分类。

SVM 的基本思想是在各类别之间找到尽可能宽的间隔。

位于间隔边缘的数据点称为支持向量，它们决定决策边界的位置。



scikit-learn 中的 SVM 示例：核技巧

```
from sklearn.pipeline import make_pipeline
from sklearn.preprocessing import StandardScaler
from sklearn.svm import SVC
from sklearn.datasets import make_moons
# Generate non-linear moon-shaped data
X, y = make_moons(n_samples=100, noise=0.15, random_state=42)
# Create a pipeline to scale data then apply a polynomial SVM
poly_svm_clf = make_pipeline(
    StandardScaler(),
    SVC(kernel="poly", degree=3, coef0=1, C=5)
)
poly_svm_clf.fit(X, y)
```

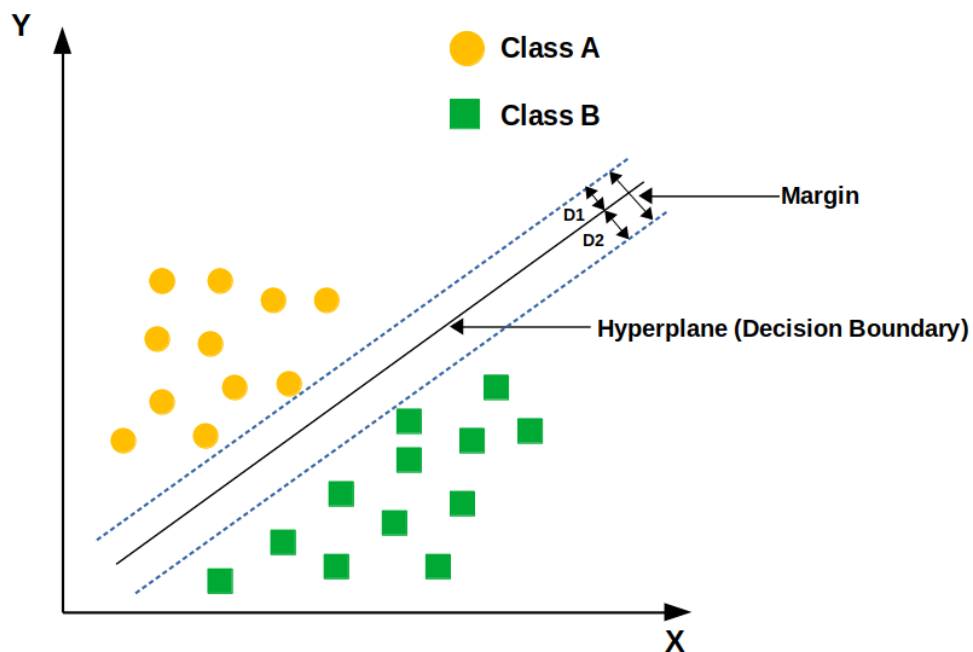
多项式核

- 对于非线性数据，一种做法是添加多项式特征。但这在计算上可能很昂贵。
- SVM 可以使用称为核技巧的数学方法，在不实际添加这些特征的情况下达到同样效果。
- 我们也可以选择其他核，例如径向基函数（RBF）核。

“C”超参数

在 scikit-learn 的 SVM 类中，'C' 超参数控制软间隔分类。

- 较小的 C 使间隔更宽，但允许更多间隔违反，对应更强的正则化。
- 较大的 C 使间隔更窄，但间隔违反更少，对应更弱的正则化，并可能导致过拟合。



数据未必能被完美地线性分开。

我们设定 C，在尽可能保持大间隔与限制间隔违反数量（落入间隔内或落在错误一侧的样本）之间取得平衡。

决策树

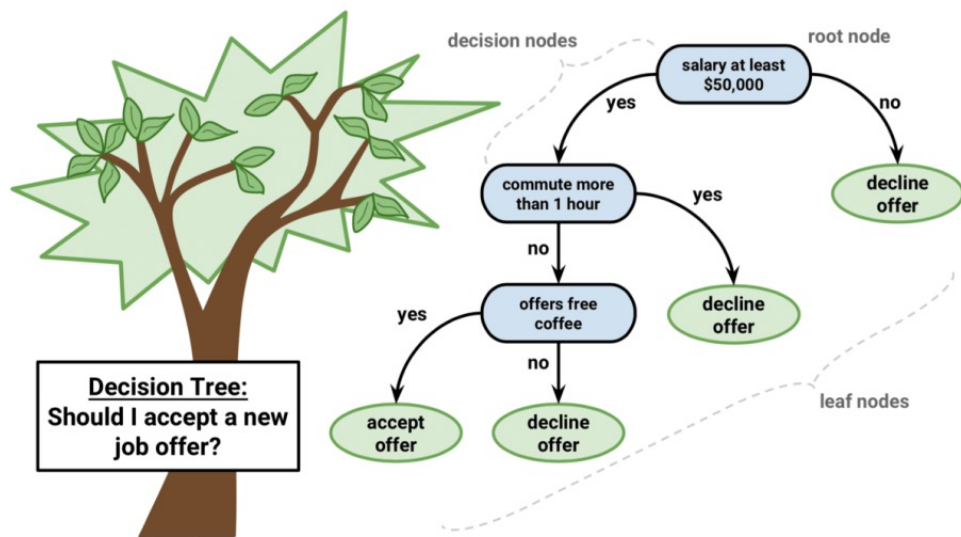
决策树是一种非参数监督学习方法，可用于分类和回归。

- 优点：可解释（“白盒”模型）、可处理数值与类别数据、无需特征缩放、对数据依赖低
- 缺点：容易过拟合、可能不稳定（数据的微小变化可能导致完全不同的树）、对某些概念处理困难（如 XOR）

决策树

决策树是一种非参数监督学习方法，可用于分类和回归。

- 优点：可解释（“白盒”模型）、可处理数值与类别数据、无需特征缩放、对数据依赖低
- 缺点：容易过拟合、可能不稳定（数据的微小变化可能导致完全不同的树）、对某些概念处理困难（如 XOR）



进行预测时，样本从根节点向下遍历至叶节点。

每个内部节点针对单个特征提出一个真/假问题。

预测为该叶中训练样本的多数类（分类）或目标均值（回归）。

scikit-learn 中的决策树示例

scikit-learn 为决策树提供两个主要类：

- 'DecisionTreeClassifier' 用于分类任务。
- 'DecisionTreeRegressor' 用于回归任务。

```
from sklearn.datasets import load_iris
from sklearn.tree import DecisionTreeClassifier
from sklearn.model_selection import train_test_split
from sklearn.metrics import accuracy_score
iris = load_iris()
X = iris.data
y = iris.target
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42)
# max_depth is a key regularization hyperparameter
tree_clf = DecisionTreeClassifier(max_depth=3, random_state=42)
tree_clf.fit(X_train, y_train)
y_pred = tree_clf.predict(X_test)
print(f"Accuracy: {accuracy_score(y_test, y_pred):.2f}")
# Accuracy: 1.00
```

scikit-learn 中的决策树示例

scikit-learn 为决策树提供两个主要类：

- 'DecisionTreeClassifier' 用于分类任务。
- 'DecisionTreeRegressor' 用于回归任务。

```
from sklearn.tree import DecisionTreeRegressor
import numpy as np

# Create some dummy regression data
X = np.array([[0, 0], [2, 2]])
y = np.array([0.5, 2.5])

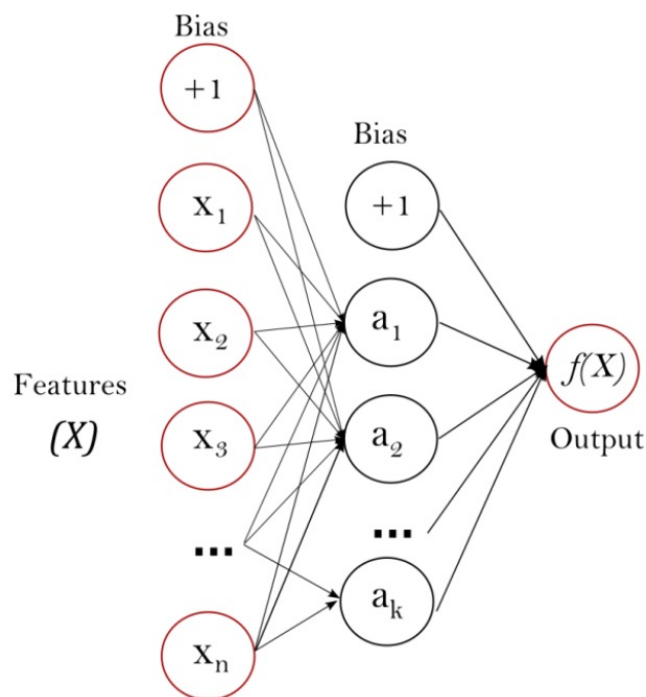
# Train the regressor
tree_reg = DecisionTreeRegressor()
tree_reg.fit(X, y)

# Predict the value for a new point
new_point = [[1, 1]]
print(tree_reg.predict(new_point)) # [0.5]
```

对新样本的预测，是其所落入叶节点中训练样本的平均目标值。

■ 多层感知机 (MLP)

多层感知机 (MLP) 是一种全连接的前馈人工神经网络。它是通用模型，能够学习复杂的非线性函数。



scikit-learn 中的 MLP 示例

- scikit-learn 提供 'MLPClassifier' 和 'MLPRegressor'。
- 对 MLP 而言，特征缩放是关键的处理步骤

```
from sklearn.neural_network import MLPClassifier
from sklearn.preprocessing import StandardScaler
from sklearn.pipeline import make_pipeline

# It is highly recommended to use a pipeline for scaling
mlp_pipeline = make_pipeline(
    StandardScaler(),
    MLPClassifier(
        hidden_layer_sizes=(100, 50), # Two hidden layers
        max_iter=1000,
        random_state=42
    )
)

mlp_pipeline.fit(X_train, y_train)
print(f"MLP Accuracy: {mlp_pipeline.score(X_test, y_test):.2f}")
```

监督学习模型

- 线性回归
- 逻辑回归
- 支持向量机 (SVM)
- 决策树
- 多层感知机 (MLP)

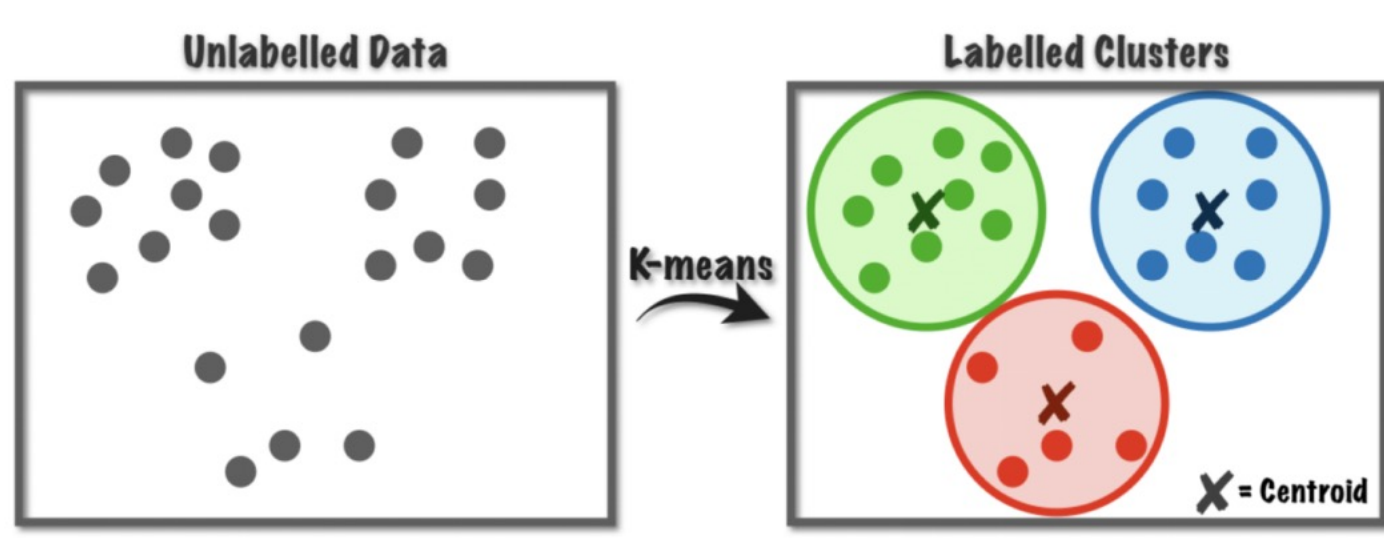
无监督学习模型 (聚类)

- K-Means
- 凝聚聚类
- DBSCAN
- 谱聚类
- 高斯混合

无监督学习：聚类

目标：在无需预先标注结果的情况下，发现数据中的隐藏模式或分组。

- scikit-learn 提供多种聚类算法。
- 算法的选择很大程度上取决于数据结构与分析目标。



K-Means 聚类

K-Means 将 n 个观测划分为 k 个簇，每个观测属于均值（簇中心或质心）最近的簇。

它通用、快速、可扩展，适合大小均匀的球形簇。

```
from sklearn.cluster import KMeans
from sklearn.datasets import load_iris
# Load the data (we ignore the true labels for clustering)
iris = load_iris()
X = iris.data

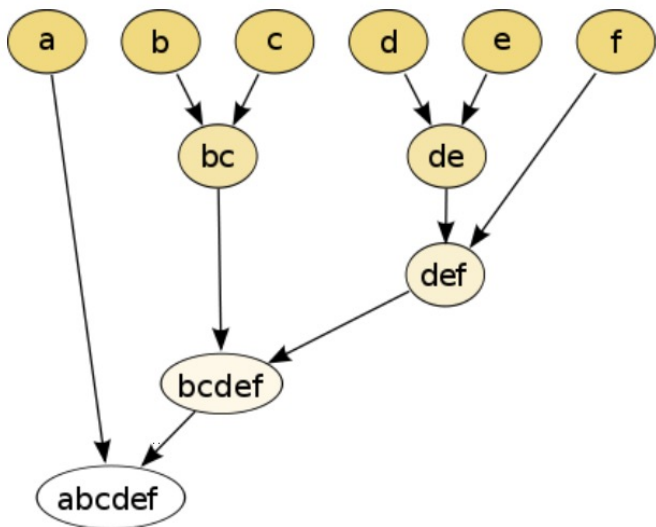
# We know there are 3 species, so we set n_clusters=3
kmeans = KMeans(n_clusters=3, n_init=10, random_state=42)
# Train the model and get cluster assignments
kmeans.fit(X)
cluster_labels = kmeans.labels_
print("Cluster assignments for the first 10 samples:")
print(cluster_labels[:10])
# Predict the cluster for new data points
new_data = [[5.1, 3.5, 1.4, 0.2], [6.7, 3.0, 5.2, 2.3]]
predictions = kmeans.predict(new_data)
print("\nPredictions for new data:", predictions)
```

凝聚聚类

一种层次聚类方法，通过不断合并较小的簇来构建嵌套簇。可使用多种距离度量

```
from sklearn.cluster import AgglomerativeClustering

clustering = AgglomerativeClustering(n_clusters=2).fit(X)
print("Agglomerative Clustering labels:", clustering.labels_)
```



■ 还有更多...

- DBSCAN：可以发现非平坦几何形状与大小不均的簇。擅长去除异常点。
- 谱聚类：适用于非平坦几何形状（如嵌套圆）。基于图距离。
- 高斯混合：一种概率模型，假设各簇来自不同的高斯分布。可用于密度估计。

■ 课堂练习

用 Titanic 数据集的前 300 个样本训练决策树，并预测其余样本中乘客是否生还。评估你的预测有多好。

- 加载完整 Titanic 数据集并选择特征（例如 'Pclass'、'Sex'、'Age'、'Fare'）。
- 预处理全部特征，以处理缺失值并编码类别。
- 将处理后的数据划分为训练集（前 300 个样本）和测试集（其余样本）。
- 在训练集上训练 'DecisionTreeClassifier'。
- 在测试集上评估其准确率。



模型评估与选择

机器学习的目标

问题：我们的真正目标是什么？

- 是在训练数据上达到 100% 准确率吗？
- 不是。靠记住训练集很容易“作弊”。
- 类比：背下 10 道练习题、却不掌握底层概念的学生，会在真正的考试中失败
- 泛化是学习的本质——将知识应用到所见例子之外的能力。

The True Goal of Generalization

Build a model that performs well on **new, unseen data** drawn from the **same distribution** as the training data.

如何形式化“表现好”？

风险最小化：

- 我们有损失函数 $\ell(f(x), y)$ ，用于衡量预测 $f(x)$ 相对真实标签 y 有多差。
- 我们希望找到使真实风险（或期望风险）最小的函数 f
- 这是真实数据分布上所有可能数据点的平均损失。

经验风险最小化 (ERM) $\mathcal{R}(f) = \mathbb{E}_{(x,y) \sim \mathcal{P}}[\ell(f(x), y)]$

- 我们不知道真实数据分布，只有有限的训练集
- 因此，我们改为最小化经验风险（训练集上的平均损失）

$$\mathcal{R}_{\text{emp}}(f) = \frac{1}{N} \sum_{i=1}^N \ell(f(x_i), y_i)$$

核心矛盾：欠拟合与过拟合

最小化经验风险（训练误差）是有用的代理，但会带来一个根本性的矛盾：

欠拟合：

- 问题：模型过于简单，无法捕捉数据中的底层模式。
- 症状：训练误差高，测试误差也高。
- 类比：完全没有学习的学生。练习和真正考试都会失败。

过拟合：

- 问题：模型过于复杂。它记住了训练数据，包括其中的噪声。
- 症状：训练误差很低，但测试误差很高。
- 类比：背下那 10 道练习题的学生。练习满分，真正考试失败。

Generalization is the "sweet spot" in between.

泛化框架：偏差与方差

我们可以形式化地分解模型的期望测试误差：

偏差：

- 来自模型“固执”或错误假设的误差。
- 在所有可能数据集上训练得到的平均模型，离真实函数有多远？
- **高偏差 → 欠拟合。（例如用线性模型拟合正弦波）**

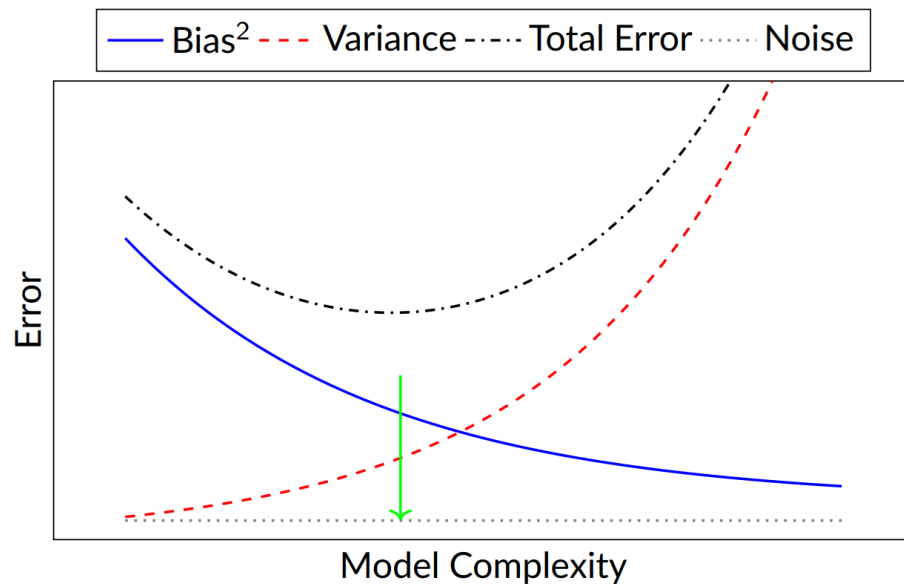
方差：

- 来自模型对其所见特定训练数据“敏感”的误差。
- 在不同数据集上训练的模型彼此差异有多大？
- **高方差 → 过拟合。（例如高次多项式为贴住每个点而剧烈波动）**

偏差-方差权衡

偏差与方差之间存在权衡，由模型复杂度控制。

- 低复杂度模型（例如线性回归）
 - 低方差（所有模型看起来相似）
 - 高偏差（假设很可能是错的）
- 高复杂度模型（例如 15 次多项式、深层神经网络）
 - 高方差（模型对数据非常敏感）
 - 低偏差（几乎可以拟合任何东西）

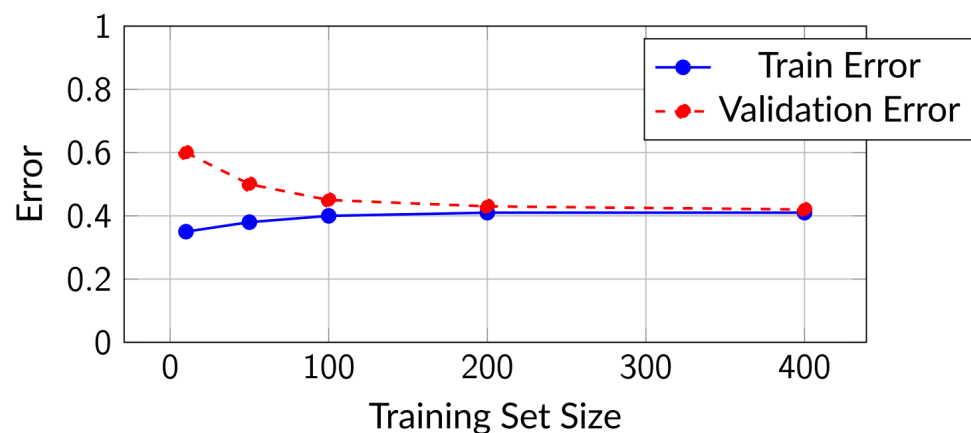


The Implementation of Generalization

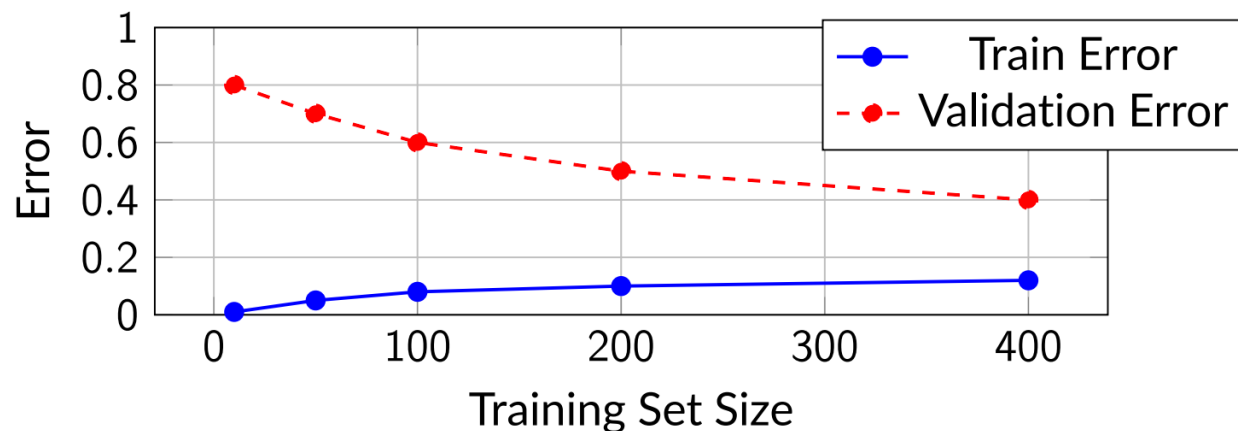
Find the "sweet spot" of complexity that minimizes the **Total Error**.

设计原则

- 把算法所学的（训练）与我们所决定的（选择）分开，并保留一份最终未被使用的测试集用于报告。
- 选择过程寻找使估计测试误差最小的复杂度或正则化水平。



低复杂度 $\Rightarrow$ 高偏差：评估应显示已收敛且仍然较高的训练/验证误差（诊断欠拟合）。

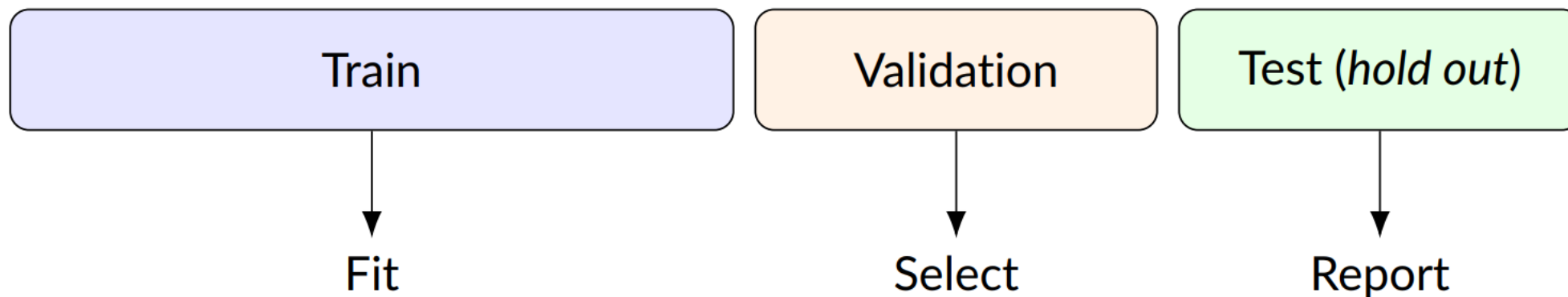


高复杂度 $\Rightarrow$ 高方差：评估应显示训练误差与验证误差之间的差距（诊断过拟合）。

■ 规范的数据划分流程

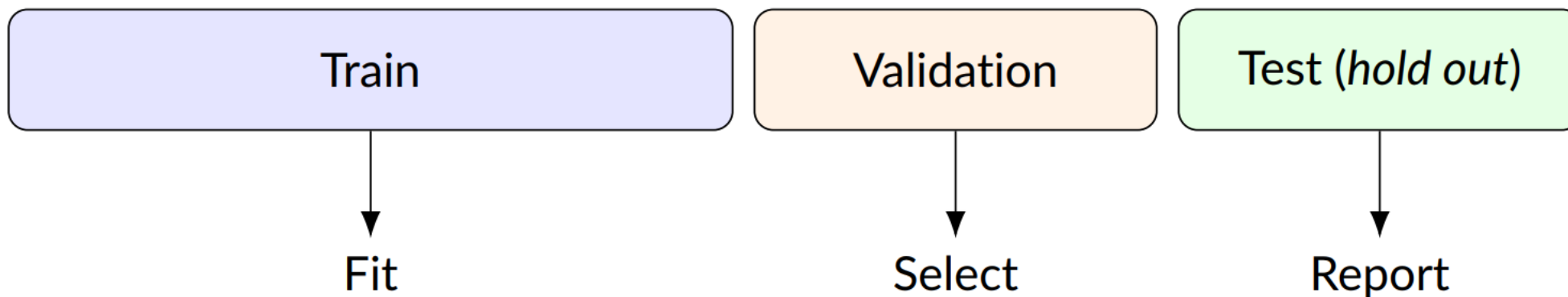
- 训练：拟合模型参数。
- 验证：选择模型类型/超参数/早停。
- 测试（仅一次）：最终的无偏估计。

划分示例：60% 训练 / 20% 验证 / 20% 测试（适用于较大的 N ）。



可能出什么问题？

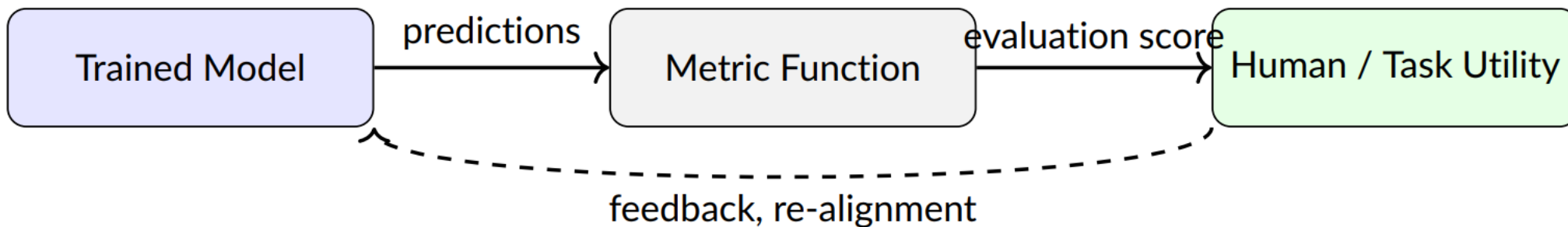
- 划分不具代表性：单次随机划分会得出脆弱的结论。
- 泄漏：在训练或调参时使用测试集信息。切勿在训练或选择过程中使用测试数据。只在最后使用一次。
- 对验证集过拟合：反复使用同一验证划分，直到它实际上变成训练集。



模型评估：为何指标重要

指标量化模型与下游任务中我们真正关心的目标有多一致。

- 用定量标准客观衡量模型行为。
- 把人或任务目标（准确率、安全性、效率、公平性、满意度）转化为可测量的信号。
- 作为改进、选择与部署决策的反馈信号。



回归:

- 大误差风险 $\Rightarrow$ MSE/RMSE
- 稳健性 $\Rightarrow$ MAE

分类

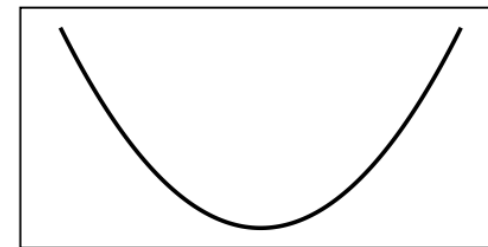
- 逻辑回归 $\Rightarrow$ Log/交叉熵损失
- 正例少 $\Rightarrow$ PR-AP
- 类别均衡 $\Rightarrow$ ROC-AUC
- 需要工作点 $\Rightarrow$ 精确率/召回率/F1

回归：MSE 与 RMSE

优点/缺点

- 优点：MSE 平滑且可微；强调大误差。
- 缺点：对异常值敏感；依赖量纲。

$$\text{MSE} = \frac{1}{N} \sum_{i=1}^N (y_i - \hat{y}_i)^2, \quad \text{RMSE} = \sqrt{\text{MSE}}$$



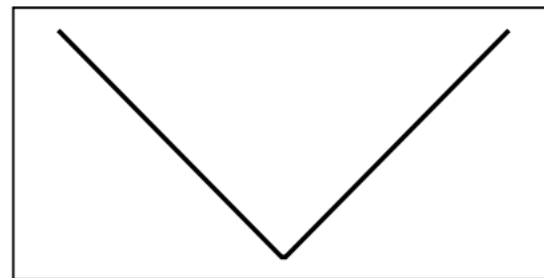
```
from sklearn.metrics import mean_squared_error, root_mean_squared_error  
mse = mean_squared_error(y_true, y_pred)  
rmse = root_mean_squared_error(y_true, y_pred)
```

■ 回归：MAE

优点/缺点

- 优点：对异常值稳健；可用原始单位解释。
- 缺点：在 0 处不光滑；对大误差不够敏感。

$$\text{MAE} = \frac{1}{N} \sum_{i=1}^N |y_i - \hat{y}_i|$$



```
from sklearn.metrics import mean_absolute_error  
mae = mean_absolute_error(y_true, y_pred)
```

分类：对数损失（用于逻辑回归）

对数损失（二元交叉熵）对自信的错误预测惩罚很重。

- 若真实标签 $y = 1$ ，代价为 $-\log(p)$ 。当 p 接近 0 时代价极大。
- 若真实标签 $y = 0$ ，代价为 $-\log(1-p)$ 。当 p 接近 1 时代价极大。

Log Loss Cost Function $J(\theta)$

$$J(\theta) = -\frac{1}{m} \sum_{i=1}^m \left[y^{(i)} \log(\hat{p}^{(i)}) + (1 - y^{(i)}) \log(1 - \hat{p}^{(i)}) \right]$$

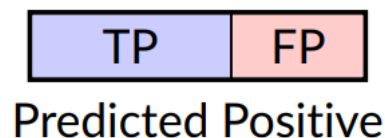
```
from sklearn.metrics import log_loss  
  
loss = log_loss(y_true, y_pred_prob)
```

分类：精确率

精确率衡量正类预测的可靠性。

- 优点：误报代价高时很有用。
- 缺点：不考虑被漏掉的正例。

$$\text{Precision} = \frac{\text{TP}}{\text{TP} + \text{FP}}$$



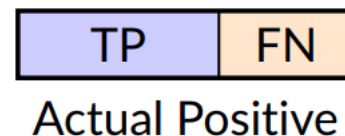
```
from sklearn.metrics import precision_score
prec = precision_score(y_true, y_pred_labels, average='
    binary')
```

分类：召回率

召回率衡量对真正例的覆盖程度。

- 优点：漏掉正例代价高时很有用；对安全/健康应用很重要。
- 缺点：不考虑假正例。

$$\text{Recall} = \frac{\text{TP}}{\text{TP} + \text{FN}}$$



```
from sklearn.metrics import recall_score
rec = recall_score(y_true, y_pred_labels, average='
    binary')
```

■ 分类: F1

F1 平衡精确率与召回率，当二者的平衡很重要时优先使用。

- 优点：用单一数值概括 PR 权衡；适用于不平衡分类。
- 缺点：忽略真负例；可能掩盖性能差来自精确率低还是召回率低。

$$F1 = 2 \cdot \frac{\text{Precision} \cdot \text{Recall}}{\text{Precision} + \text{Recall}}$$

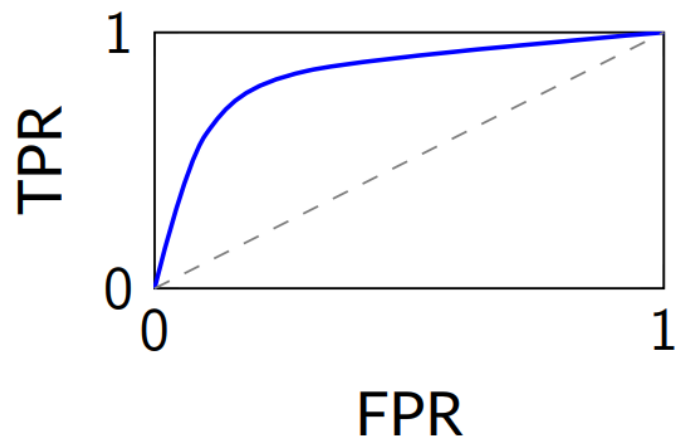
```
from sklearn.metrics import f1_score  
f1 = f1_score(y_true, y_pred_labels, average='binary')
```

分类：ROC-AUC

ROC-AUC 衡量排序质量，适合在选定阈值之前先追求良好排序。

- 优点：不依赖阈值；对类别不平衡较不敏感。
- 缺点：严重不平衡时，可能掩盖正类上的较差表现。

$$\text{TPR} = \frac{\text{TP}}{\text{TP} + \text{FN}}, \quad \text{FPR} = \frac{\text{FP}}{\text{FP} + \text{TN}}$$



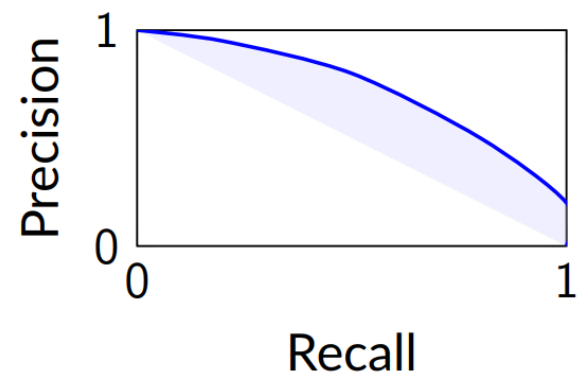
```
from sklearn.metrics import roc_auc_score, roc_curve
auc = roc_auc_score(y_true, y_scores)
fpr, tpr, thr = roc_curve(y_true, y_scores)
```

分类：PR-AP

PR-AP 概括不同阈值下的 精确率-召回率 权衡，尤其当正类稀少时。

- 优点：关注正类表现；适用于不平衡任务。
- 缺点：对类别占比敏感；部署时仍需确定阈值。

$$AP = \int_0^1 \text{Precision}(r) dr$$



```
from sklearn.metrics import average_precision_score,
    precision_recall_curve
ap = average_precision_score(y_true, y_scores)
prec, rec, thr = precision_recall_curve(y_true, y_scores
    )
```

回归:

- 大误差风险 $\Rightarrow$ MSE/RMSE
- 稳健性 $\Rightarrow$ MAE

分类

- 逻辑回归 $\Rightarrow$ Log/交叉熵损失
- 正例少 $\Rightarrow$ PR-AP
- 类别均衡 $\Rightarrow$ ROC-AUC
- 需要工作点 $\Rightarrow$ 精确率/召回率/F1

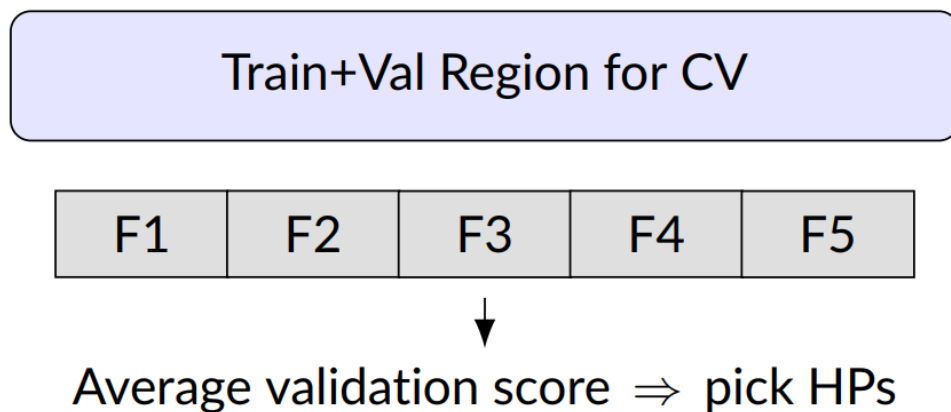
始终

- 报告置信区间 (CI) 或 CV (交叉验证) $\text{mean} \pm \text{std}$
- 明确给出阈值

模型选择：核心流程

目标：选出预期泛化最好的模型/超参数。

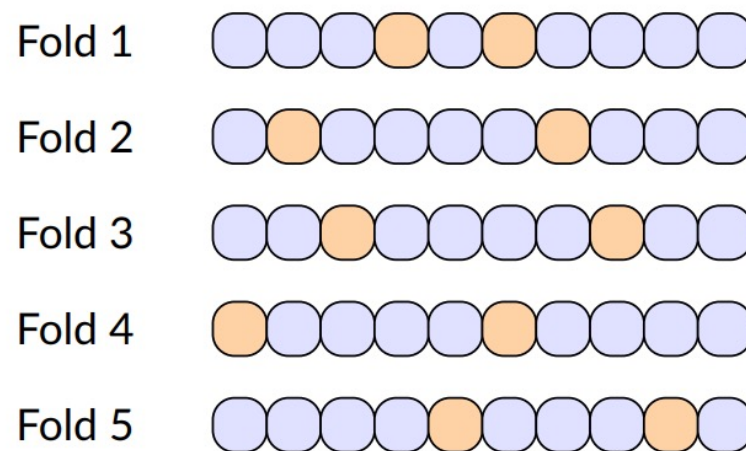
- 定义搜索空间（模型类型、超参数（HP）范围）
- 在训练+验证部分用 K 折交叉验证（CV）估计每种设置的性能
- 选择平均 CV 分数最好的设置
- 用所选配置在训练+验证上重新训练
- 在测试集上评估一次



- 分层 K 折 - 各折保持类别比例
- 分组 K 折 - 各组在各折之间保持分离
- 时间序列划分 - 保持时间顺序
- 嵌套交叉验证 - 在模型选择之后给出无偏评估

■ 分层 K 折

各折保持类别比例；降低选择信号的方差。

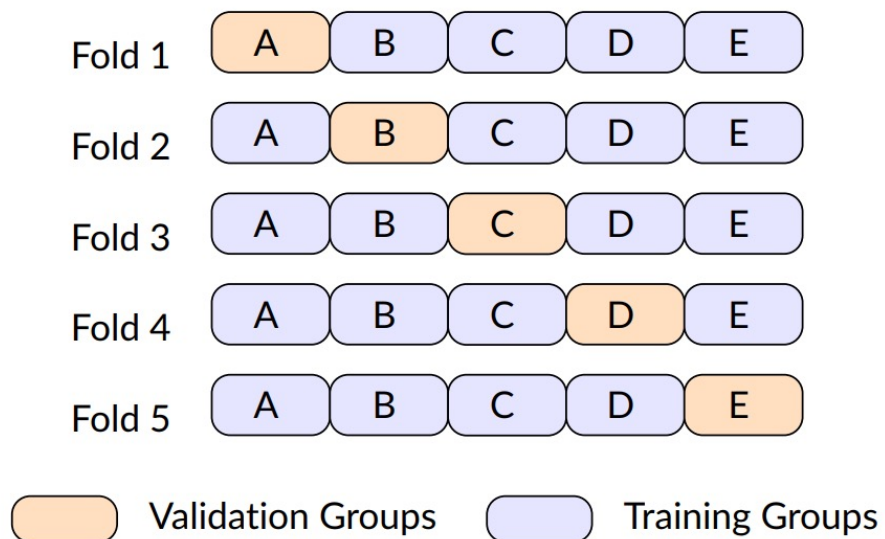


 Positive Class (minority)  Negative Class (majority)

```
from sklearn.model_selection import StratifiedKFold,
    cross_val_score
skf = StratifiedKFold(n_splits=5, shuffle=True,
    random_state=42)
scores = cross_val_score(model, X, y, cv=skf, scoring='
    f1')
```

■ 分组 K 折

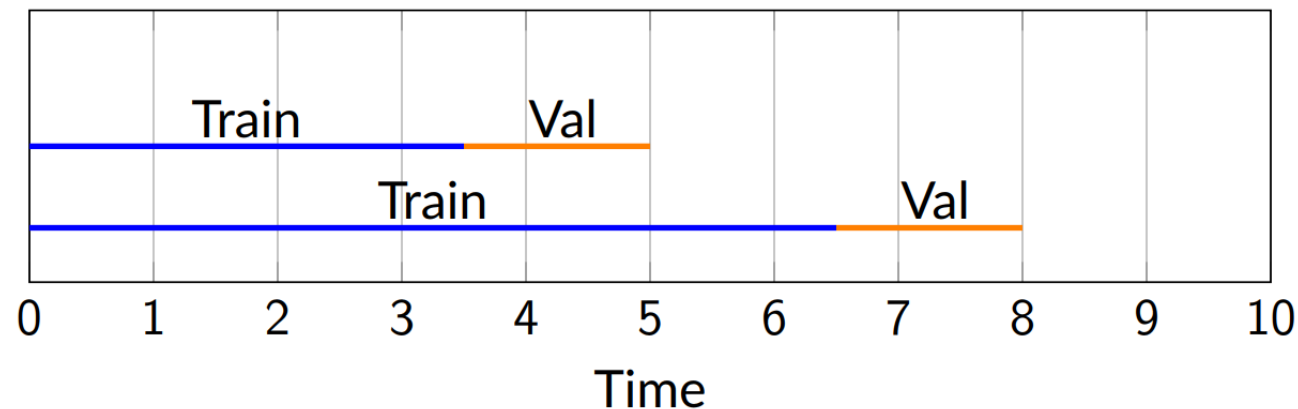
将同一组（例如患者/用户）的全部样本放在同一折；避免乐观偏差。



```
from sklearn.model_selection import GroupKFold,
    cross_val_score
# groups: array-like indicating entity id per sample
gkf = GroupKFold(n_splits=5)
scores = cross_val_score(model, X, y, groups=groups, cv=
    gkf, scoring='roc_auc')
```

时间序列划分

保持时间顺序：用过去数据训练，用未来数据验证。
使用滚动窗口或扩展窗口；不要打乱。



```
from sklearn.model_selection import TimeSeriesSplit,
    cross_val_score
tscv = TimeSeriesSplit(n_splits=5)
scores = cross_val_score(model, X, y,
                          cv=tscv, scoring='
                          neg_root_mean_squared_error')

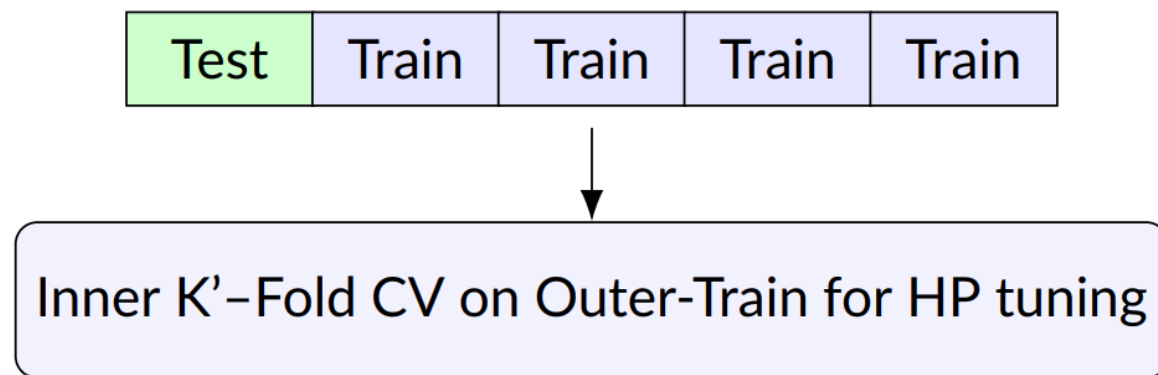
rmse = -scores.mean()
print(rmse)
```

■ 嵌套交叉验证（用于无偏性能估计）

问题：用同一套 CV 结果既做模型选择又做最终报告，会导致过于乐观的有偏估计

嵌套 CV:

- 外层循环（K 折）：评估折。
- 内层循环（K'）：在每个外层训练划分内做选择与调参。
- 报告外层测试分数的 $\text{mean} \pm \text{std}$ 。





超参数优化

■ 什么是超参数？

超参数是训练前指定、且不从数据中学习的配置选择。

类型：

- 连续：学习率、权重衰减。
- 离散：估计器数量、层数、近邻数。
- 类别：优化器选择、核类型、激活函数。
- 条件：仅当另一选择成立时才生效的超参数（例如 `kernel=rbf` 时再调 γ ）

■ 为何做超参数（HP）优化？

超参数影响模型容量、正则化与训练动态。

- 精度与稳健性：HP 改变偏差-方差；选得好才能到达泛化的“最佳平衡点”。
- 效率：训练时间、内存与推理成本取决于 HP。
- 可靠性：稳定、可复现的性能需要有原则的调参协议。
- 公平比较：各方法须在可比的调参预算下比较。

考虑模型参数 θ 、超参数 λ

清晰分离

- 用训练集为每个 λ 拟合参数 θ 。
- 用验证集（或 CV）选择 λ 。
- 选定后在测试集上评估一次。

数据有限时

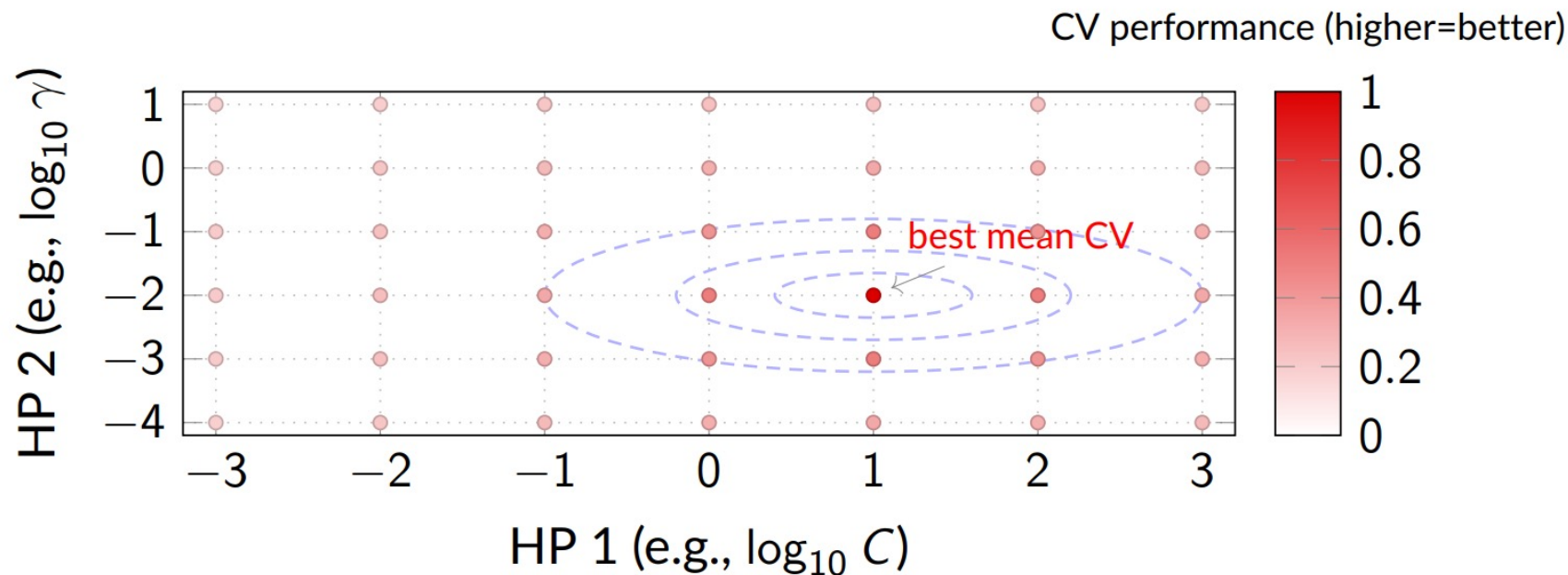
- 用 K 折 CV 获得更稳定的选择信号。
- 用嵌套 CV 获得偏差更小的性能估计。

- 网格搜索
- 随机搜索
- 逐次减半

网格搜索

思路：穷尽评估预先定义的超参数组合。

- 优势：确定、可复现、易于并行；适合较小的搜索空间。
- 劣势：搜索空间大时代价高；可能在不重要的超参数上浪费试验。

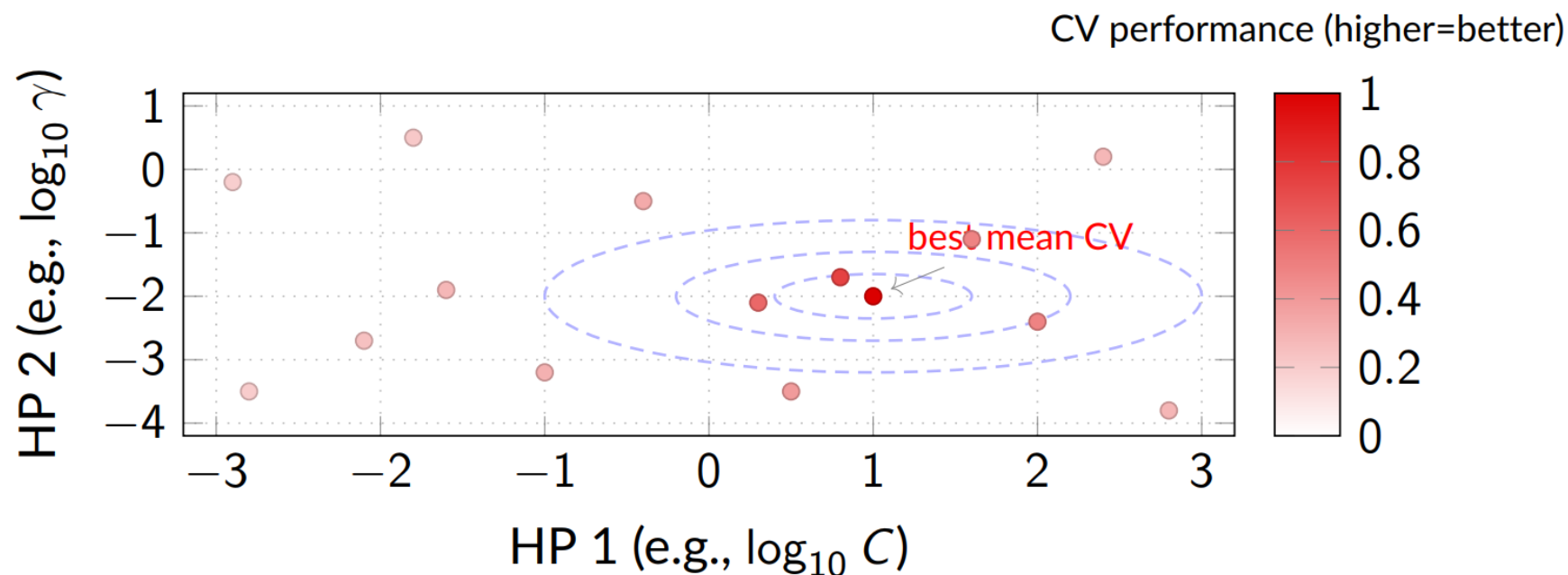


```
from sklearn import svm, datasets
from sklearn.model_selection import GridSearchCV,
    train_test_split
# Load example data
X, y = datasets.load_iris(return_X_y=True)
X_train, X_test, y_train, y_test = train_test_split(X, y
    , test_size=0.3, random_state=42)
# Define model and hyperparameter grid
svc = svm.SVC()
param_grid = {'C': [0.1, 1, 10, 100], 'gamma': [1, 0.1,
    0.01, 0.001], 'kernel': ['rbf']}
# Perform grid search with 5-fold cross-validation
grid = GridSearchCV(svc, param_grid, refit=True, cv=5,
    verbose=0)
grid.fit(X_train, y_train)
print("Best parameters:", grid.best_params_)
print("Best CV accuracy:", grid.best_score_)
print("Test accuracy:", grid.score(X_test, y_test))
```

随机搜索

思路：不逐一评估每个网格点，而是从超参数空间中随机采样配置。

- 优势：在高维空间中更高效；用更少评估即可找到较好配置。
- 劣势：结果可能因随机性而变化；可能错过虽小但重要的区域。

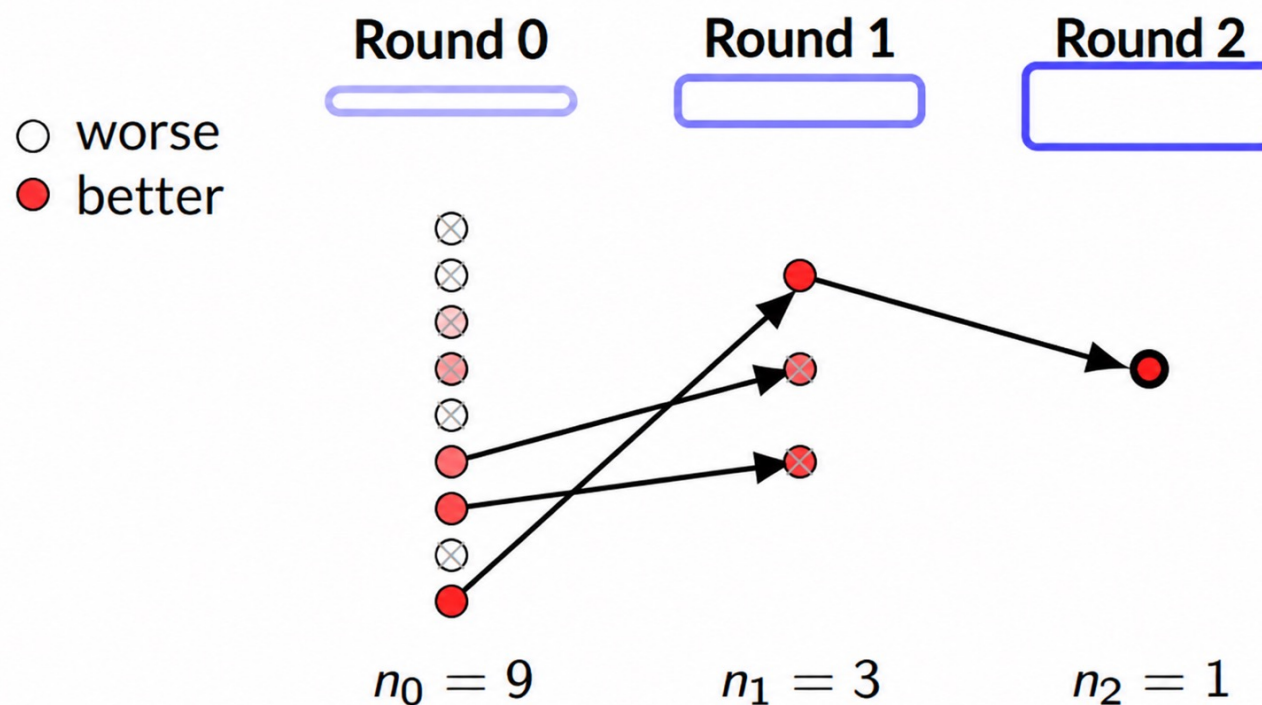


scikit-learn 中的随机搜索示例

```
from sklearn import svm, datasets
from sklearn.model_selection import RandomizedSearchCV,
    train_test_split
from scipy.stats import loguniform
# Load example data
X, y = datasets.load_iris(return_X_y=True)
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size
    =0.3, random_state=42, stratify=y)
# Define model and hyperparameter distributions
svc = svm.SVC()
param_distributions = {'C': loguniform(1e-2, 1e2), 'gamma': loguniform
    (1e-4, 1e0), 'kernel': ['rbf']}
# Perform random search with 5-fold cross-validation
rand = RandomizedSearchCV(estimator=svc, param_distributions=
    param_distributions, n_iter=20, cv=5, random_state=42, n_jobs=-1,
    verbose=0)
rand.fit(X_train, y_train)
print("Best parameters:", rand.best_params_)
print("Best CV accuracy:", rand.best_score_)
print("Test accuracy:", rand.score(X_test, y_test))
```

逐次减半

思路：以较小预算启动大量配置；每轮保留较优者并为其分配更多预算。
适用情形：搜索空间大、预算有限，且早期表现能预测最终表现。



scikit-learn 中的 逐次减半 示例

```
from sklearn.experimental import enable_halving_search_cv
from sklearn.model_selection import HalvingGridSearchCV,
    HalvingRandomSearchCV, train_test_split
from sklearn import svm, datasets
from scipy.stats import loguniform
X, y = datasets.load_iris(return_X_y=True)
Xtr, Xte, ytr, yte = train_test_split(X, y, test_size=0.3,
    random_state=42, stratify=y)
svc = svm.SVC()
param_dist = {'C': loguniform(1e-2, 1e2), 'gamma': loguniform(1e-4, 1
    e0), 'kernel': ['rbf']}
shr = HalvingRandomSearchCV(estimator=svc, param_distributions=
    param_dist, factor=3, resource='n_samples', max_resources='auto',
    min_resources=50, random_state=42, n_candidates='exhaust', scoring
    ='accuracy')
shr.fit(Xtr, ytr)
print("SH best params:", shr.best_params_, "CV acc:", shr.best_score_)
print("Test acc:", shr.score(Xte, yte))
```

■ 搜索空间设计

好的搜索空间与搜索算法同样重要。

- 尺度：为不同超参数使用合适的尺度（例如学习率用对数尺度）。
- 范围：根据先验知识与试运行选择合理范围。
- 结构：编码依赖关系以避免无效配置（例如仅对 RBF 核调节 γ ）。
- 覆盖：平衡探索与效率：过宽会浪费预算；过窄可能错过好的配置。

目标：用 逐次减半 随机搜索调节 SVC。

固定设置

- 数据集：sklearn.datasets.breast_cancer，用于二分类。
- 划分：70% 训练集 / 30% 测试集，分层，random_state=42。
- 估计器：Pipeline(StandardScaler(), SVC(kernel='rbf', probability=True))。
- 优化器：sklearn.model_selection.HalvingRandomSearchCV, cv=StratifiedKFold(5, shuffle=True, random_state=42), scoring='roc_auc', n_jobs=-1, random_state=42。
- 搜索空间：C $\in$ [10⁻², 10²], gamma γ $\in$ [10⁻⁴, 100], 二者均按对数均匀采样。
- 逐次减半设置：使用 factor=3、resource='n_samples'，让搜索把更多训练样本分配给有希望的配置。

需要实现

1. 在训练集划分上拟合 HalvingRandomSearchCV。
2. 用最佳超参数在完整训练集划分上重新拟合 pipeline。
3. 在测试集划分上评估一次。
4. 打印：最佳超参数；每次减半迭代中评估的候选数；最佳交叉验证 ROC-AUC，即所选配置在最终减半迭代上 5 折的 mean $\pm$ std；测试集 ROC-AUC

Scikit-Learn 机器学习基础

- 加载数据集、划分数据
- 数据预处理：缩放、归一化、编码、离散化、二值化、填补、生成多项式特征、降维

Scikit-Learn 中的模型

- 监督学习模型：线性/逻辑回归、SVM、决策树、MLP
- 无监督学习模型：K-Means、凝聚聚类、...

模型评估与选择

- 泛化：偏差-方差权衡
- 数据划分协议
- 评估指标：MSE、RMSE、MAE、log loss、ROC-AUC、PR-AP、精确率/召回率/F1
- 交叉验证：分层 K 折、分组 K 折、时间序列划分、嵌套交叉验证

超参数优化

- 搜索算法：网格搜索、随机搜索、逐次减半
- 搜索空间设计

Python 编程与人工智能

感谢聆听!